Can FrodoKEM Run in a Millisecond? FPGA Says Yes!

Gökçe Düzyol[1,2], Muhammed Said Gündoğan[1] and Atakan Arslan[1]

[1] TÜBİTAK BİLGEM National Research Institute of Electronics and Cryptology, Kocaeli, Türkiye {gokce.duzyol,said.gundogan,atakan.arslan}@tubitak.gov.tr
[2] Electrical and Electronics Engineering Department, Boğaziçi University, İstanbul, Türkiye

Abstract.
FrodoKEM is a post-quantum key encapsulation mechanism based on plain Learning With Errors (LWE). In contrast to module-lattice-based schemes, it relies on an unstructured variant of the LWE problem, providing more conservative and better-understood security guarantees. As a result, FrodoKEM has been recommended by European cybersecurity agencies such as BSI and ANSSI, and has also been proposed in international standardization efforts, including ISO and the IETF Internet-Draft process.
In this paper, we explore hardware-level parallelization techniques for FrodoKEM. To date, the only notable attempt to parallelize FrodoKEM in hardware was made by Howe et al. [HMOR21] in 2021. In their work, the SHAKE function was identified as a performance bottleneck and replaced by the Trivium stream cipher. However, this replacement renders the implementation incompatible with standardized recommendations. In contrast, our work adheres strictly to the original FrodoKEM specification, including its use of SHAKE as the PRNG, and introduces a scalable architecture enabling high-throughput parallel execution.
For FrodoKEM-640, we present parallel architectures for key generation, encapsulation, and decapsulation. Our implementation achieves between 976 and 1077 operations per second, making it the fastest FrodoKEM hardware implementation reported to date. Furthermore, we propose a general architecture that offers a scalable area-throughput trade-off: by increasing the number of DSPs and proportionally scaling BRAM usage, our design can be scaled to achieve significantly higher performance beyond the reported implementation. This demonstrates that SHAKE is not inherently a barrier to parallel matrix multiplication, and that efficient, standard-compliant FrodoKEM implementations are achievable for high-speed cryptographic applications.

Keywords: FrodoKEM · Key Encapsulation Mechanism · Hardware Acceleration · FPGA · Post-quantum Cryptography

1 Introduction

The development of scalable quantum computing will make it possible to solve problems that are currently beyond the reach of classical computers. While this progress benefits more fields, it also poses a significant threat to modern cryptographic infrastructures. Public-key cryptosystems, such as RSA and ECC, rely on the hardness of integer factorization and the discrete logarithm problem. However, a quantum computer running Shor's algorithm [Sho94] could break these schemes in polynomial time, rendering them ineffective for secure communications, digital signatures, and authentication protocols. This vulnerability endangers critical infrastructures, including financial transactions, secure messaging, and government communications, necessitating the development of quantum-resistant cryptographic alternatives.

To address the threat posed by quantum computers, NIST launched the Post-Quantum Cryptography (PQC) Standardization project in 2016. After three rounds of evaluation, NIST selected four algorithms for standardization: three digital signature schemes and one public-key encryption scheme—CRYSTALS-KYBER—which was standardized under the name ML-KEM [Nat24].

While the module lattice-based scheme Kyber has been selected as the primary standard, the alternative candidate FrodoKEM continues to attract significant interest. FrodoKEM distinguishes itself by relying on standard (unstructured) Learning With Errors (LWE), offering conservative security guarantees without relying on algebraic structures such as rings or modules. This makes FrodoKEM a preferred choice for applications that prioritize long-term robustness against unforeseen cryptanalytic advances.

Interest in FrodoKEM has extended beyond the NIST process. Several European countries, including Germany, France, and the Netherlands, have expressed support for conservative schemes like FrodoKEM and Classic McEliece. In 2022, ISO/IEC began work on standardizing FrodoKEM, and by April 2023, it was accepted into the ISO/IEC 18033-2 standard. Most recently, in March 2025, it was submitted to the IRTF CFRG as an Internet-Draft [LBES25].

FrodoKEM is designed based on the standard LWE problem, and comes with a trade-off: performance. By avoiding algebraically structured lattices such as those used in RLWE or MLWE-based schemes, FrodoKEM sacrifices efficiency in terms of key and ciphertext sizes and computational cost compared to schemes like Kyber.

While previous works have explored memory optimizations or software-based side-channel protections, FrodoKEM's performance in high-throughput hardware environments—particularly on FPGA-based accelerators—has not been thoroughly investigated. In this work, we present a matrix multiplication architecture that parallelizes computation to accelerate FrodoKEM on FPGAs, aiming to meet high-throughput demands.

Contributions.

The most time-consuming operation in the FrodoKEM algorithm is matrix multiplication. During key generation, an $8 \times n$ matrix S is multiplied by an $n \times n$ matrix A, while during encapsulation and decapsulation, the $n \times n$ matrix A is multiplied by an $n \times 8$ matrix S'. Here, the parameter n takes values of 640, 976, and 1344 for the security levels corresponding to Categories 1, 3, and 5, respectively.

In the $S \cdot A$ multiplication, the matrix S can be viewed as 8 vectors, enabling an 8-fold vector-matrix multiplication parallelization. Similarly, the $A \cdot S'$ operation can be parallelized as an 8-fold matrix-vector multiplication. Achieving further speedup, however, becomes challenging. Although the rows of matrix A can be generated independently, the generation of each individual row using the SHAKE function is inherently sequential and cannot be parallelized.

This limitation was also discussed in [HMOR21] , where the authors proposed replacing SHAKE with the Trivium stream cipher as the XOF to enable 16× and 14× parallelism in FrodoKEM. In contrast, our work explores parallelism in FrodoKEM's matrix multiplications while strictly adhering to the original specification that uses SHAKE.

In Section 5, we propose two hardware architectures: a parallel $k \times 8$ architecture for $A \cdot S$ multiplication, and a parallel $8 \times n \times m$ architecture for $S' \cdot A$ multiplication. These architectures scale the number of multiplications per clock cycle with the number of DSPs employed and require a proportional amount of BRAM. This yields a flexible area-throughput trade-off that can scale to high-throughput demands.

Contrary to common belief, we demonstrate that the use of SHAKE as a PRNG does not prevent the parallelization of matrix multiplication. Our architectures achieve high throughput while remaining fully compliant with the original FrodoKEM specification that uses SHAKE.

As an architectural realization, we provide an implementation of FrodoKEM-640-SHAKE, presented in Section 6, targeting a latency of 1 ms. To achieve this goal, we employ a 3×8 architecture (corresponding to a 24× speedup) for key generation and an $8 \times 2 \times 2$ architecture (32× speedup) for encapsulation and decapsulation.

Our implementation achieves 976–1077 operations per second, making it the fastest FrodoKEM hardware implementation reported to date. Furthermore, the proposed architecture enables even higher performance through a scalable area-throughput trade-off, supporting further parallelization given sufficient hardware resources. This demonstrates that FrodoKEM can be a practical option for high-throughput cryptographic applications.

Tables 1, 2, and 3 compare our implementation against existing FrodoKEM hardware designs. Among prior works, only the implementation by Howe et al. [HMOR21] achieves comparable throughput, but it deviates from the FrodoKEM specification by replacing SHAKE with an alternative, non-standard-compliant PRNG.

Although our implementation specifically targets FrodoKEM-640-SHAKE, the proposed architecture is general-purpose and readily extensible to support FrodoKEM-976-SHAKE and FrodoKEM-1344-SHAKE. These higher-security variants can also benefit from the same parallelization strategies, enabling acceleration proportional to the available hardware resources.

2 Related Work

In 2018, Howe et al. presented an FPGA implementation as well as a software implementation of FrodoKEM-640 and FrodoKEM-976, the latter targeting the ARM Cortex-M4F platform [HOKG18]. The authors propose an FPGA design that optimizes the balance between area consumption and throughput by using a single LWE multiplication core while executing other operations in parallel, ensuring constant-time execution. Their FPGA implementation achieves a full key exchange in 60 ms at the 128-bit security level and 135 ms at the 192-bit level.

In 2021, Buschkowski et al. [Bus21] investigated the efficiency of High-Level Synthesis (HLS) methods in comparison to direct hardware implementations. The authors referenced the design principles outlined in [HOKG18] for the direct hardware implementation approach. Given the inherently complex and time-consuming nature of direct hardware design, the authors aimed to achieve comparable performance with reduced effort through HLS. The study involved a comparison between direct hardware implementation and optimized HLS method. The results indicated that the optimized HLS method delivered performance comparable to that of the direct implementation, particularly in terms of latency and resource utilization.

In 2021, Howe et al. [HMOR21] identify SHAKE, a commonly used seed-expanding function, as a major performance bottleneck in hardware implementations of FrodoKEM. They also considered SHAKE a limiting factor for achieving parallel and high-throughput designs, due to its inherently sequential structure. To address this, the authors propose replacing SHAKE with Trivium, a lightweight stream cipher that offers higher throughput and lower area consumption. The work primarily optimizes FrodoKEM by parallelizing matrix multiplication, a key operation in its cryptographic computations. By leveraging Trivium's efficiency, the researchers achieve significant speed-ups while maintaining a similar FPGA area footprint. Their optimized hardware implementation results in a 16x speed-up for encapsulation and a 14x speed-up for decapsulation compared to previous designs. Additionally, they incorporate first-order masking for the decapsulation module, ensuring resistance against power analysis attacks.

While this study presents an optimized implementation of FrodoKEM by replacing `SHAKE` with Trivium to enhance performance, it deviates from the original FrodoKEM specification. Although the use of Trivium offers advantages in terms of throughput and area

efficiency, it may raise concerns regarding compatibility with standardized implementations. This is particularly relevant, as many future cryptographic systems and security modules are expected to conform to the original FrodoKEM specification—recommended by the German [Fed25], French [Fre23], and Dutch [Net23] governments, as well as by ISO and IETF—all of which favor AES or SHAKE for randomness generation.

In 2023, Bos et al. [BBC+23] investigated the implementation of FrodoKEM on resource-constrained devices by reducing the algorithm's memory footprint. They presented results suitable for integration into software libraries with minimal modifications. One of the key optimizations in their work involves generating the matrix A on-the-fly, which reduces memory usage but introduces the overhead of repeated recomputation. In contrast, the design presented in this work also generates matrix A on-the-fly, but does so only once, thereby avoiding redundant computations.

3 Preliminary

This section provides a brief overview of the FrodoKEM algorithm, with a focus on the components relevant to our FPGA-based implementation.

3.1 FrodoKEM

FrodoKEM is a post-quantum key encapsulation mechanism (KEM) whose security relies on the Learning With Errors (LWE) problem over unstructured lattices. Unlike structured lattice-based schemes such as Ring-LWE and Module-LWE, FrodoKEM avoids any algebraic structure, offering a conservative approach with strong theoretical robustness against quantum adversaries. The scheme provides IND-CCA security at three levels corresponding to the NIST Post-Quantum Cryptography call for proposals. FrodoKEM-640 targets Level 1 security, offering protection comparable to AES-128. FrodoKEM-976 corresponds to Level 3, with a security level similar to AES-192, while FrodoKEM-1344 targets Level 5, equivalent to AES-256 [LBES25].

FrodoKEM consists of two main variants: a standard variant (simply called FrodoKEM), which imposes no restrictions on key reuse and is suitable for applications where many ciphertexts may be encrypted under a single public key, and an ephemeral variant (eFrodoKEM), which is designed for use cases involving only a small number of ciphertexts per key. The main difference between these variants lies in the presence of a salt in the standard version, while eFrodoKEM omits this component. This distinction has a negligible impact on performance. Since these two variants are virtually identical from a performance perspective, we refer to both under the common name FrodoKEM throughout this paper without explicitly distinguishing between them.

Due to the absence of algebraic structure in FrodoKEM, storing and transmitting the public matrix A directly would incur significant overhead. To address this, a seed is embedded in the key pair and expanded into matrix A at the beginning of each algorithm execution using the generic function `Frodo.Gen`, as shown in Algorithms 1, 2, and 3. This function can be instantiated with either AES-128 or SHAKE-128; while AES-128 is typically used in software-based implementations, SHAKE-128 is generally preferred in hardware due to its superior efficiency. Algorithm 4 illustrates the use of SHAKE-128 for matrix A generation.

FrodoKEM supports three parameter sets corresponding to different security levels. FrodoKEM-640 targets NIST security Level 1, offering protection comparable to AES-128; FrodoKEM-976 targets Level 3, with security similar to AES-192; and FrodoKEM-1344 targets Level 5, providing security equivalent to AES-256 [LBES25].

The key generation, encapsulation, and decapsulation algorithms for eFrodoKEM-640-SHAKE are presented in Algorithm 1, Algorithm 2, and Algorithm 3, respectively. A key

component in all three algorithms is the generation of the matrix A, which is constructed using the SHAKE-128 function to produce a matrix $A \in \mathbb{Z}_q^{n \times n}$. The algorithm responsible for this generation is shown in Algorithm 4.

The structure of the eFrodoKEM implementation is largely identical to that of FrodoKEM; with minor modifications, the same implementation can be adapted to support FrodoKEM. This implementation follows the official specification [ABD+24], which defines the eFrodoKEM algorithm.

Algorithm 1 FrodoKEM.KeyGen

function FrodoKEM.KeyGen
  $s \| \text{seed}_{SE} \| z \leftarrow \mathcal{U}(\{0,1\}^{\text{len}_s + \text{len}_{SE} + \text{len}_z})$
  $\text{seed}_A \leftarrow \text{SHAKE}(z, \text{len}_{\text{seed}_A})$
  $A \in \mathbb{Z}_q^{n \times n}$ via $A \leftarrow \text{Frodo.Gen}(\text{seed}_A)$
  $(r^{(0)}, r^{(1)}, \ldots, r^{(2\bar{n}n-1)}) \leftarrow \text{SHAKE}(0x5F \| \text{seed}_{SE}, 2\bar{n}n \cdot \text{len}_\chi)$
  $S^T \leftarrow \text{Frodo.SampleMatrix}((r^{(0)}, r^{(1)}, \ldots, r^{(\bar{n}n-1)}), \bar{n}, n, T_\chi)$
  $E \leftarrow \text{Frodo.SampleMatrix}((r^{(n\bar{n})}, r^{(n\bar{n}+1)}, \ldots, r^{(2n\bar{n}-1)}), n, \bar{n}, T_\chi)$
  $B \leftarrow AS + E$
  $b \leftarrow \text{Frodo.Pack}(B)$
  $pkh \leftarrow \text{SHAKE}(\text{seed}_A \| b, \text{len}_{pkh})$
  return $pk \leftarrow \text{seed}_A \| b, \quad sk' \leftarrow (s \| \text{seed}_A \| b, S^T, pkh)$
end function

Algorithm 2 FrodoKEM.Encaps

function FrodoKEM.Encaps(pk)
  Parse $pk = (\text{seed}_A \| b)$
  $\mu \leftarrow \mathcal{U}(\{0,1\}^{\text{len}_\mu})$
  $pkh \leftarrow \text{SHAKE}(pk, \text{len}_{pk})$
  $(\text{seed}_{SE} \| k) \leftarrow \text{SHAKE}(pkh \| \mu, \text{len}_{seed_{SE}} + \text{len}_k)$
  $(r^{(0)}, r^{(1)}, \ldots, r^{(2\bar{m}n + \overline{\bar{m}\bar{n}}-1)}) \leftarrow \text{SHAKE}(0x96 \| \text{seed}_{SE}, (2\bar{m}n + \overline{\bar{m}\bar{n}}) \cdot \text{len}_\chi)$
  $S' \leftarrow \text{Frodo.SampleMatrix}((r^{(0)}, r^{(1)}, \ldots, r^{(\bar{m}n-1)}), \bar{m}, n, T_\chi)$
  $E' \leftarrow \text{Frodo.SampleMatrix}((r^{(\bar{m}n)}, r^{(\bar{m}n+1)}, \ldots, r^{(2\bar{m}n-1)}), \bar{m}, n, T_\chi)$
  $A \leftarrow \text{Frodo.Gen}(\text{seed}_A)$
  $B' \leftarrow S'A + E'$
  $c_1 \leftarrow \text{Frodo.Pack}(B')$
  $E'' \leftarrow \text{Frodo.SampleMatrix}((r^{(2\bar{m}n)}, r^{(2\bar{m}n+1)}, \ldots, r^{(2\bar{m}n + \overline{\bar{m}\bar{n}}-1)}), \bar{m}, \bar{n}, T_\chi)$
  $B \leftarrow \text{Frodo.Unpack}(b, n, \bar{n})$
  $V \leftarrow S'B + E''$
  $C \leftarrow V + \text{Frodo.Encode}(\mu)$
  $c_2 \leftarrow \text{Frodo.Pack}(C)$
  $\text{ss} \leftarrow \text{SHAKE}(c_1 \| c_2 \| k, \text{len}_{ss})$
  return $(c_1 \| c_2, \text{ss})$
end function

3.2 Field Programmable Gate Arrays

A Field Programmable Gate Array (FPGA) is a reconfigurable integrated circuit that can be programmed post-manufacturing, unlike Application-Specific Integrated Circuits (ASICs), which are fixed at fabrication time. This flexibility makes FPGAs ideal for prototyping, design testing, iterative development and small-scale production whereas ASICs are more suited for finalized, large-scale production.

Algorithm 3 FrodoKEM.Decaps

function FrodoKEM.Decaps($c_1 \| c_2, sk'$)
  Parse $sk' = (s \| \text{seed}_A \| b, S^T, pkh)$
  $B' \leftarrow \text{Frodo.Unpack}(c_1, \bar{m}, n)$
  $C \leftarrow \text{Frodo.Unpack}(c_2, \bar{m}, \bar{n})$
  $M \leftarrow C - B'S$
  $\mu' \leftarrow \text{Frodo.Decode}(M)$
  Parse $pk = (\text{seed}_A \| b)$
  $(\text{seed}'_{SE} \| k') \leftarrow \text{SHAKE}(pkh \| \mu', \text{len}_{seed_{SE}} + \text{len}_k)$
  $(r^{(0)}, r^{(1)}, \ldots, r^{(2\bar{m}n+\overline{mn}-1)}) \leftarrow \text{SHAKE}(0x96 \| \text{seed}'_{SE}, (2\bar{m}n + \overline{mn}) \cdot \text{len}_\chi)$
  $S' \leftarrow \text{Frodo.SampleMatrix}((r^{(0)}, r^{(1)}, \ldots, r^{(\bar{m}n-1)}), \bar{m}, n, T_\chi)$
  $E' \leftarrow \text{Frodo.SampleMatrix}((r^{(\bar{m}n)}, r^{(\bar{m}n+1)}, \ldots, r^{(2\bar{m}n-1)}), \bar{m}, n, T_\chi)$
  $A \leftarrow \text{Frodo.Gen}(\text{seed}_A)$
  $B'' \leftarrow S'A + E'$
  $E'' \leftarrow \text{Frodo.SampleMatrix}((r^{2\bar{m}n}, r^{(2\bar{m}n+1)}, \ldots, r^{(2\bar{m}n+\overline{mn}-1)}), \bar{m}, \bar{n}, T_\chi)$
  $B \leftarrow \text{Frodo.Unpack}(b, n, \bar{n})$
  $V \leftarrow S'B + E''$
  $C' \leftarrow V + \text{Frodo.Encode}(\mu')$
  if $B \| C = B'' \| C'$ **then**
    return $ss \leftarrow \text{SHAKE}(c_1 \| c_2 \| k', \text{len}_{ss})$
  else
    return $ss \leftarrow \text{SHAKE}(c_1 \| c_2 \| s, \text{len}_{ss})$
  end if
end function

FPGAs are categorized into families that share architectural characteristics but differ in performance and capacity. For instance, the Xilinx Artix-7 family offers a low-cost, resource-efficient platform suitable for embedded applications, while higher-end families like Virtex-7 cater to performance-intensive use cases. Each FPGA contains a variety of configurable resources—such as logic blocks, DSP slices, and memory—that enable implementation of custom hardware functionality.

Modern FPGAs consist of various configurable resources, each tailored to perform specific tasks efficiently. Understanding these resources is essential for designing optimized hardware architectures, particularly in cryptographic applications such as PQC.

- **Flip-Flops (FFs):** FFs are fundamental storage elements used to hold single-bit values. While they offer the fastest data access times, their limited quantity makes them unsuitable for storing large datasets.

Algorithm 4 Frodo.Gen using SHAKE-128

Require: Seed $\texttt{seedA} \in \{0,1\}^{\text{len}_{\texttt{seedA}}}$
Ensure: Pseudorandom matrix $A \in \mathbb{Z}_q^{n \times n}$
for $i = 0$ **to** $n - 1$ **do**
  $b \leftarrow \langle i \rangle \| \texttt{seedA} \in \{0,1\}^{16+\text{len}_{\texttt{seedA}}}$
  $\langle c_{i,0} \rangle \| \langle c_{i,1} \rangle \| \cdots \| \langle c_{i,n-1} \rangle \leftarrow \texttt{SHAKE128}(b, 16n)$ where each $\langle c_{i,j} \rangle \in \{0,1\}^{16}$
  for $j = 0$ **to** $n - 1$ **do**
    $A_{i,j} \leftarrow c_{i,j} \bmod q$
  end for
end for
return A

- **Look-Up Tables (LUTs):** LUTs are the primary logic elements in FPGAs, capable of realizing arbitrary boolean functions by storing truth tables. Typically supporting 5 to 6 input bits, multiple LUTs can be combined for complex operations. They can also be repurposed as distributed memory (LUTRAM), though with slower access than FFs. Due to resource constraints, LUTRAMs are less efficient for storing large data structures like cryptographic keys.

- **Block RAMs (BRAMs):** BRAMs are specialized memory blocks integrated into FPGAs, designed for efficient storage of large data sets. In Xilinx devices, a typical BRAM has a capacity of 36 Kbits, which can either function as a single memory unit or be split into two 18 Kbit blocks. These blocks support various configurations for data width and depth, enabling flexible use—from storing thousands of single-bit values to handling wider data elements (e.g., 64 or 128 bits) by combining multiple BRAMs.

  BRAMs support several access modes that determine the number of simultaneous read and write operations. In single-port mode, one read and one write can occur sequentially in a clock cycle. Simple dual-port mode adds a second read port, while true dual-port mode allows two reads, two writes, or one read and one write per cycle. This versatility makes BRAMs highly suitable for performance-critical applications like post-quantum cryptography, where both speed and storage capacity are crucial.

- **DSP Blocks:** Digital Signal Processing (DSP) units enable high-speed arithmetic operations, particularly multiply-accumulate (MAC) computations. These are crucial for operations like matrix or polynomial multiplication and offer significant performance advantages over LUT-based implementations.

The availability and capacity of these resources vary significantly across FPGA families. While entry-level devices typically offer limited logic and memory, high-performance families provide substantially greater capabilities.

4 Challenges in Accelerating Matrix Multiplication in FrodoKEM

In hardware implementations of the FrodoKEM algorithm, the most time-consuming and performance-critical operation is the multiplication of matrices over $\mathbb{Z}_q$. This operation occurs in Step 8 of Algorithm 1 (`FrodoKEM.KeyGen`), Step 10 of Algorithm 2 (`FrodoKEM.Encaps`), and Step 13 of Algorithm 3 (`FrodoKEM.Decaps`), and it dominates the overall computation time. Based on our observations for the FrodoKEM-640-SHAKE parameter set, the matrix multiplication step accounts for approximately 99.58%, 98.49%, and 97.38% of the total clock cycles during key generation, encapsulation, and decapsulation, respectively [Bus21]. Therefore, this section focuses on the main challenges in optimizing this step to improve the overall hardware performance of FrodoKEM.

Following the approach described in [Bus21], matrix multiplication is implemented as a series of sequential matrix-vector or vector-matrix multiplications.Specifically, the matrix-vector multiplication involves the matrix A as the left operand and the column vectors of S, resulting in $A \cdot S$. In contrast, the vector-matrix multiplication uses the row vectors of S' as the left operand and the matrix A as the right operand, yielding $S' \cdot A$. A natural way to improve parallelism is to increase the number of hardware multipliers. However, this is not a straightforward modification. Several architectural constraints and implementation challenges must be addressed before such scaling is feasible. These are summarized below:

- The BRAMs used in this work are implemented as true dual-port blocks, where the number of simultaneous read and write accesses is limited to two per block. As a result, when multiple parallel multipliers attempt to access memory concurrently, this limitation can become a bottleneck to data throughput.

- The SHAKE function generates the elements of matrix S column by column, and the elements of S', E, and E' row by row, each in a strictly sequential manner. Due to this inherent sequentiality, parallelizing the generation of these matrices is not feasible. Therefore, the most practical approach is to generate and store them in advance.

- Matrix A occupies approximately 819,200 kilobytes of memory, making it impractical to store entirely on the FPGA. As a result, it is generated dynamically, one row at a time. While one row of A is used for multiplication, the next row is generated in parallel. This process is implemented using a *double-buffering scheme*, in which two memory buffers alternate between read and write operations. As described in [HOKG18], this method enables partial generation and use of the matrix without requiring full in-memory storage. Specifically, while one BRAM buffer holds and feeds the current row of A to the multiplier, the next row is simultaneously generated and written to the other buffer. Once complete, the roles of the buffers are swapped, ensuring continuous data availability for the matrix multiplication process.

- In the matrix multiplication $A \cdot S$ during key generation, the matrix S consists of 8 columns. If these columns are stored separately as individual vectors, the multiplication can be performed as 8 parallel matrix-vector multiplications, enabling an 8-fold speedup. To achieve even greater acceleration, each matrix-vector multiplication can be further sped up by a factor of k, resulting in an overall speedup of $k \times 8$, as presented in the Section 5.1 .

- During the Encapsulation and Decapsulation procedures, the matrix multiplication takes the form $S' \cdot A$, where S' is the left operand. However, matrix A is still generated row-by-row, as in the Key Generation phase. Therefore, the row-column multiplication approach used in Key Generation is not directly applicable here. The acceleration method for this case is presented in Section 5.3.

5 Proposed Hardware Acceleration Methods for Matrix Multiplication

Assuming that the matrix S is precomputed and appropriately stored, the multiplication $A \cdot S$ can be viewed as eight independent matrix-vector multiplications, one for each column of S. Similarly, the multiplication $S' \cdot A$ corresponds to eight parallel vector-matrix multiplications, where each row of S' is multiplied by matrix A. However, existing literature does not provide acceleration methods beyond this 8-fold parallelism. The main reason lies in the fact that, although the rows of matrix A can be generated independently, the generation of each individual row using the SHAKE function is inherently sequential and cannot be parallelized.

This limitation has also been highlighted in [HMOR21], where the authors propose replacing SHAKE with the Trivium stream cipher as the XOF to enable 16× and 14× parallelism in FrodoKEM. In contrast, this section explores how further acceleration can be achieved while strictly adhering to the original FrodoKEM specification that uses SHAKE.

5.1 Parallel $k \times l \times 8$ Architecture for AS Multiplication

In this section, we describe a hardware architecture that enables each matrix-vector multiplication in the $A \cdot S$ operation to be further parallelized by a factor of $k \times l$, in addition to the inherent 8-fold parallelism due to the 8 columns of matrix S. This results in an overall parallelism factor of $8kl$, allowing the deployment of $8kl$ hardware multipliers (DSPs) and achieving a corresponding speedup, provided that supporting resources are available.

As previously discussed, matrix A is generated on-the-fly using a double-buffering scheme. When the row of A to be multiplied is stored in a single storage unit which is synthesized to be 0.5 BRAM block, true dual-port access allows two simultaneous reads, which is sufficient for $k = 2$. However, for higher parallelism levels such as $k = 3$ or $k = 4$, both the active row of matrix A and the column vectors of matrix S must be distributed across multiple storage units to support the required number of read accesses. Specifically, for a given k, the current row of A is partitioned across $\lceil \frac{k}{2} \rceil$ storage units, and each of the 8 columns of S is similarly stored using $\lceil \frac{k}{2} \rceil$ storage units. This organization enables full parallel feeding of $8k$ multipliers.

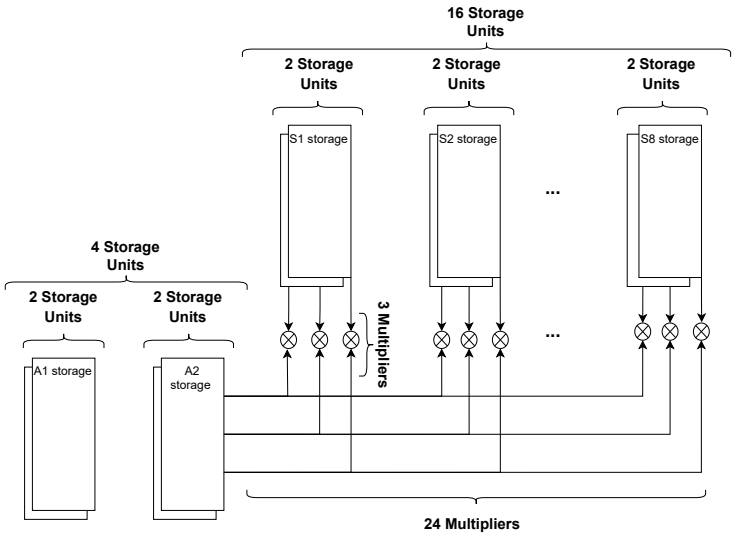

Figure 1: Parallel $k \times l \times 8$ Architecture Instantiated with $k = 3$ and $l = 1$ for $A \cdot S$ Multiplication

As k increases, it is essential to consider whether the SHAKE function can keep up with the data consumption rate. In the double-buffering scheme, while one row is being used in the matrix multiplication process with $8k$ multipliers, the next row must be generated using SHAKE. For example, in FrodoKEM-640, one row contains 640 elements; hence, the row multiplication with $8k$ multipliers completes in $\frac{640}{k}$ clock cycles. If SHAKE cannot generate the next row within this time, it becomes the performance bottleneck and limits the effectiveness of the $8k$ parallelism.

To address this, the first solution is to use a highly optimized SHAKE implementation. However, for large values of k, this alone may be insufficient. A more scalable solution involves deploying multiple SHAKE generators, each operating with its own double-buffering unit, and generating l rows of matrix A in parallel into these buffers. This approach increases the resource requirement by a factor of l, while providing l-fold

parallelism. Consequently, a total of $8kl$ multiplications can be performed per clock cycle using $8kl$ DSP units.

In our implementation, we employed an efficient SHAKE design, and observed that up to $k = 3$, SHAKE did not constitute a bottleneck. Therefore, without requiring additional SHAKE units (i.e., $l = 1$), we were able to perform key generation in 1.024 ms. The implementation details are discussed in Section 6.1. The $k \times l \times 8$ architecture used in the implementation, with $k = 3$ and $l = 1$, is illustrated in Figure 1.

5.2 Right-Side Multiplication in Encapsulation and Decapsulation

In key generation, the matrix multiplication follows $B = AS + E$, whereas in encapsulation we have

$$B' = S'A + E', \tag{1}$$

and similarly in decapsulation,

$$B'' = S'A + E'. \tag{2}$$

These operations require a different and more clever approach than the one used in key generation, since the matrix A appears on the right-hand side of the multiplication. While matrix A is generated on-the-fly row by row using SHAKE, the multiplication involves the rows of S' and the columns of A. To address this challenge, we adopt the method proposed in [Bus21], which will be explained in this section. Since the encapsulation and decapsulation multiplications are structurally similar, the encapsulation and decapsulation processes will be addressed jointly in a single section. For clarity, we drop the prime notation and expand the equation in full matrix form:

$$\begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1,640} \\ b_{21} & b_{22} & \cdots & b_{2,640} \\ \vdots & \vdots & \ddots & \vdots \\ b_{8,1} & b_{8,2} & \cdots & b_{8,640} \end{bmatrix} = \begin{bmatrix} s_{11} & s_{12} & \cdots & s_{1,640} \\ s_{21} & s_{22} & \cdots & s_{2,640} \\ \vdots & \vdots & \ddots & \vdots \\ s_{8,1} & s_{8,2} & \cdots & s_{8,640} \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1,640} \\ a_{21} & a_{22} & \cdots & a_{2,640} \\ \vdots & \vdots & \ddots & \vdots \\ a_{640,1} & a_{640,2} & \cdots & a_{640,640} \end{bmatrix} + \begin{bmatrix} e_{11} & e_{12} & \cdots & e_{1,640} \\ e_{21} & e_{22} & \cdots & e_{2,640} \\ \vdots & \vdots & \ddots & \vdots \\ e_{8,1} & e_{8,2} & \cdots & e_{8,640} \end{bmatrix} \tag{3}$$

The element-wise computation of the B' and B'' matrices can be expressed as:

$$b_{i,j} = \sum_{k=1}^{640} s_{i,k} \cdot a_{k,j} + e_{i,j} \tag{4}$$

Unlike the naive approach, where the first row of S is multiplied with the first column of A, in our case, the on-the-fly row-wise generation of A means its columns are not immediately or simultaneously available. Therefore, the multiplication must proceed based on the rows of A as they are generated. When the k-th row of matrix A is generated, its elements must be immediately used in Equation (4) wherever they are required. To enable this, the matris S and the output matrix B is stored in BRAMs, and when the k-th row of A is available, each element $b_{i,j}$ of B is updated using the operation:

$$b_{i,j} = b_{i,j} + s_{i,k} \cdot a_{k,j} \tag{5}$$

Effectively, Equation (4) is parallelized across all $b_{i,j}$ elements, and each time a row of matrix A is generated, all corresponding $b_{i,j}$ values are updated accordingly.

This approach is also applicable when m rows of matrix A are generated in parallel. In this case, the value of $b_{i,j}$ is updated by adding

$$s_{i,k} \cdot a_{k,j} + \cdots + s_{i,k+m-1} \cdot a_{k+m-1,j} \tag{6}$$

An example for $m = 2$ is provided in Algorithm 5.

5.3 Parallel $8 \times n \times m$ Architecture for $S'A$ Multiplication

Since matrix S has 8 columns, the multiplication $S' \cdot A$ can be parallelized into 8 vector-matrix multiplications. To achieve further parallelism, there are two main strategies:

- Increase the number of elements from each row of A that are multiplied, denoted by n.
- Increase the number of rows of A involved in the multiplication, denoted by m.

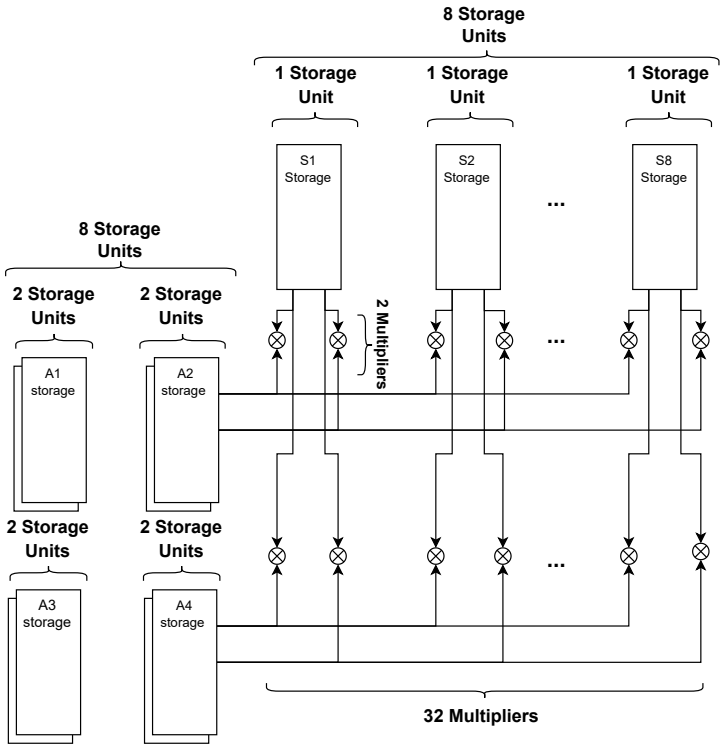

Figure 2: Parallel $8 \times n \times m$ Architecture Instantiated with $n = m = 2$ for $S' \cdot A$ Multiplication

Using n elements from a row of matrix A in each cycle enables an n-fold speedup. However, since these elements must be fetched from BRAM, and due to the limited input-output bandwidth of BRAMs, this can create a memory bottleneck. To overcome this limitation, the A matrix must be distributed across multiple storage units, scaling linearly with n, each capable of two accesses per cycle. In addition, the storage requirements for matrix B' also increase proportionally with n. This increase is due to the simultaneous read and write operations required for each element of B' during accumulation.

This introduces an area-throughput trade-off: the design achieves an approximate n-fold acceleration at the cost of using $\lceil n/2 \rceil$ times as many BRAM for storing matrix A, and n times as many BRAM for storing matrix B'.

However, increasing n is eventually constrained by the throughput of the SHAKE function, which may become a bottleneck. Therefore, n has an upper bound determined by the speed of SHAKE. To enable even greater parallelism, the second strategy is also employed: generating multiple rows of matrix A in parallel.

Generating and storing m rows of A in parallel using a double-buffering scheme increases the BRAM requirement linearly with m, but allows an m-fold speedup. This introduces another area-throughput trade-off, where achieving m-times acceleration requires m-times as many BRAM for matrix A and $\lceil m/2 \rceil$-times as many BRAM for matrix S'. As the number of rows of A involved in the multiplication increases, the number of s elements required from each row of S' also increases. To sustain the necessary data throughput, matrix S' must be distributed across additional BRAM blocks.

In total, the design performs $8 \cdot n \cdot m$ multiplications per clock cycle, resulting in a speedup factor of $8nm$ for the matrix multiplication. Accordingly, the required number of DSPs scales as $8nm$, and the BRAM usage also grows linearly with $8nm$. This architecture thus provides a scalable area-throughput trade-off, enabling significantly higher performance.

As mentioned in Section 6.2, our implementation uses the $8 \times n \times m$ architecture instantiated with $n = m = 2$. This architecture is illustrated in Figure 2.

6 An Architecture Realization for FrodoKEM-640-SHAKE

This section outlines the design decisions and implementation details of our approach. While the design is not inherently device-specific, the target platform for implementation is the *Xilinx Artix-7* FPGA.

6.1 Key Generation Implementation Details

The key generation procedure involves the $A \cdot S$ matrix multiplication as its most computationally intensive operation. A 24× speed-up is found to be sufficient to meet the desired performance. This is achieved through an 3×8 parallelization strategy, using the methodology described in Section 5.1 with the parameters chosen as $k = 3$ and $l = 1$.

- In the $A \cdot S$ multiplication, the matrix S is viewed as eight separate vectors, allowing for 8-fold matrix-vector multiplication. This architectural choice provides an 8× speed-up by enabling all eight multiplications to be performed in parallel using a single row of matrix A. As a result, each row of A needs to be generated only once. This eliminates redundant regeneration of the same row of A for each column of S, thereby reducing computational overhead.

  To support this parallelization scheme, the columns of S (and similarly the matrix E) must be precomputed and stored in advance, as they are required to be accessible throughout the computation.

- To achieve a 3× improvement in element-wise multiplication throughput (with parameter $k = 3$), three elements from a column of S and three elements from the corresponding row of A are processed in parallel. However, due to the dual-port access limitation of BRAMs, each column of S and each row of A are partitioned across two BRAM blocks—one holding even-indexed elements and the other holding odd-indexed ones. This allows simultaneous access to all required elements.

- To meet the throughput requirements for $k = 3$, the SHAKE module's outputs were buffered, enabling overlap between squeezing and round operations and thus reducing A-row generation latency.

- Since the parameter $l = 1$ is used, one element is computed per column of the output matrix in each clock cycle. Consequently, eight results are produced in parallel, requiring eight concurrent write operations. These are accommodated using four dual-port `B_storage` memory units.

6.2 Encapsulation & Decapsulation Implementation Details

The encapsulation and decapsulation procedures both share the same $S' \cdot A$ matrix multiplication as their most computationally intensive operation. As a result, the same set of optimizations applies to both, and they are addressed jointly in this section.

In the encapsulation and decapsulation procedures, a 32× speed-up is sufficient to meet the required execution time. This speed-up is achieved in three stages: 8×, 2×, and 2× corresponding to the parameter choices $n = 2$ and $m = 2$ as described in Section 5.3. These values result in the lowest BRAM utilization among configurations achieving the same speed-up, requiring a total of 20 BRAMs. The pseudocode for the 32× accelerated matrix multiplication is provided in Algorithm 5. The addition of the matrix E' is omitted from the pseudocode, as it is sufficient to perform this addition once during the traversal of rows and columns.

Algorithm 5 Compute B' and B'' matrices

Input: S' matrix, A matrix
Output: B' matrix and B'' matrix
for $i = 1 : 2 : 7$ **do** ▷ Rows of A
  for $j = 1 : 2 : 639$ **do** ▷ Columns of A
    $b_{1,j} \leftarrow s_{1,i} \cdot a_{i,j} + s_{1,i+1} \cdot a_{i+1,j} + b_{1,j}$
    $b_{2,j} \leftarrow s_{2,i} \cdot a_{i,j} + s_{2,i+1} \cdot a_{i+1,j} + b_{2,j}$
    $b_{3,j} \leftarrow s_{3,i} \cdot a_{i,j} + s_{3,i+1} \cdot a_{i+1,j} + b_{3,j}$
    $b_{4,j} \leftarrow s_{4,i} \cdot a_{i,j} + s_{4,i+1} \cdot a_{i+1,j} + b_{4,j}$
    $b_{5,j} \leftarrow s_{5,i} \cdot a_{i,j} + s_{5,i+1} \cdot a_{i+1,j} + b_{5,j}$
    $b_{6,j} \leftarrow s_{6,i} \cdot a_{i,j} + s_{6,i+1} \cdot a_{i+1,j} + b_{6,j}$
    $b_{7,j} \leftarrow s_{7,i} \cdot a_{i,j} + s_{7,i+1} \cdot a_{i+1,j} + b_{7,j}$
    $b_{8,j} \leftarrow s_{8,i} \cdot a_{i,j} + s_{8,i+1} \cdot a_{i+1,j} + b_{8,j}$

    $b_{1,j+1} \leftarrow s_{1,i} \cdot a_{i,j+1} + s_{1,i+1} \cdot a_{i+1,j+1} + b_{1,j+1}$
    $b_{2,j+1} \leftarrow s_{2,i} \cdot a_{i,j+1} + s_{2,i+1} \cdot a_{i+1,j+1} + b_{2,j+1}$
    $b_{3,j+1} \leftarrow s_{3,i} \cdot a_{i,j+1} + s_{3,i+1} \cdot a_{i+1,j+1} + b_{3,j+1}$
    $b_{4,j+1} \leftarrow s_{4,i} \cdot a_{i,j+1} + s_{4,i+1} \cdot a_{i+1,j+1} + b_{4,j+1}$
    $b_{5,j+1} \leftarrow s_{5,i} \cdot a_{i,j+1} + s_{5,i+1} \cdot a_{i+1,j+1} + b_{5,j+1}$
    $b_{6,j+1} \leftarrow s_{6,i} \cdot a_{i,j+1} + s_{6,i+1} \cdot a_{i+1,j+1} + b_{6,j+1}$
    $b_{7,j+1} \leftarrow s_{7,i} \cdot a_{i,j+1} + s_{7,i+1} \cdot a_{i+1,j+1} + b_{7,j+1}$
    $b_{8,j+1} \leftarrow s_{8,i} \cdot a_{i,j+1} + s_{8,i+1} \cdot a_{i+1,j+1} + b_{8,j+1}$
  end for
end for

- The 8× speed-up is achieved, similar to the Key Generation phase, by precomputing the entire S' matrix and storing it row-wise in distinct `S_storage` units. This allows the simultaneous computation of all 8 rows of the B' matrix.
- An additional 2× speed-up is achieved by employing a second SHAKE module for generating rows of matrix A. This corresponds to setting the parameter $m = 2$, as defined in Section 5.3. By generating two rows of A simultaneously and fetching the corresponding column of S' values, the column index can be incremented by 2 in each iteration, thereby improving throughput.
- A final 2× speed-up is achieved by computing two columns of B' in parallel. This is enabled by selecting the parameter $n = 2$, as defined in Section 5.3. In this configuration, two elements from a single row of A are used along with the same s elements from a row of S', eliminating the need to fetch additional s values.

This allows two output elements in the corresponding row of B' to be computed simultaneously, effectively doubling the column-wise computation rate.

- A distinct BRAM unit is required for each B' element being computed in parallel. This is because one port is used to read the accumulated value from the previous clock cycle, while the other port is used to write the updated value after adding the current product. Since 16 distinct B' elements are computed in each clock cycle, 16 separate storage units are needed. Each of these units utilizes approximately 0.5 BRAMs.

7 Results

In this section, we detail the results achieved through the implementation of our FrodoKEM architecture. Tables 1, 2 and 3 respectively summarize the performance metrics for the key generation, encapsulation, and decapsulation modules. The proposed design was developed using Vivado 2023.1. Consistent with the other works listed in the comparison tables, all benchmarks were targeted to the Xilinx Artix-7 platform.

Table 1 summarizes the resource utilization and performance of the FrodoKEM-640 key generation module. Our implementation employs a 24× parallel architecture, significantly reducing latency while remaining fully compliant with the FrodoKEM specification. Compared to prior standards-compliant designs [Bus21, HOKG18], which operate in the 19–20 ms range, our design reduces the latency to 1.024 ms. Although this improvement comes at the cost of increased resource usage, it enables significantly higher throughput and demonstrates the scalability of our design. In contrast, the implementation in [HMOR21] achieves comparable performance using a 16× architecture to reach 1.19 ms latency; however, this is done by replacing SHAKE with an alternative PRNG, thereby breaking compatibility with the official FrodoKEM specification.

Table 2 presents the encapsulation performance and resource utilization. Our 32× parallel design achieves an execution time of 0.928 ms, with increased resource usage compared to prior works. Designs in [Bus21] and [HOKG18] require nearly 20 ms, and even the 16× variant of [HMOR21], which replaces SHAKE with Trivium, does not match our throughput. This highlights the scalability of our SHAKE-compliant parallelization strategy.

Table 3 illustrates the decapsulation results. Similar to encapsulation, our implementation employs a 32× accelerated architecture, reducing latency to 0.95 ms while utilizing 32 DSPs and 20 BRAMs. Since the work of [HMOR21] also provides a BRAM-based implementation specifically for decapsulation, we include it in our comparison. Prior standards-compliant designs [Bus21, HOKG18] require nearly 20 ms, and even the fastest variant in [HMOR21], which abandons SHAKE in favor of Trivium, achieves only 1.31 ms—still unable to match the throughput of our fully compliant design.

These results confirm the scalability and efficiency of our architecture across all major FrodoKEM operations. Beyond being the first literature example to achieve execution times around 1 ms while fully adhering to the original FrodoKEM specification, our proposed matrix multiplication parallelization architecture enables scaling to significantly higher speeds.

8 Conclusion

In this work, we introduce a technique to accelerate FrodoKEM by parallelizing the matrix multiplication while strictly adhering to the original specification. An example implementation for FrodoKEM-640 demonstrates speeds of 976–1077 operations per second. Additionally, we propose an architecture that enables parallelization by generating the

Table 1: Resource and performance comparison of FrodoKEM-640 key generation
SC: Standard Compliance

	SC	**LUT**	**FF**	**Slices**	**DSP/ BRAM**	**Freq./Period** (MHz/ns)	**Ops/sec**	Duration (ms)	Cycle Count
This Work	✓	12,246	8,859	3793	24/18	147/6.80	976	1.024	150,533
[Bus21]	✓	12,687	6,719	-	1/6	167/5.998	51	19.74	3,291,149
[HOKG18]	✓	3,771	1,800	1,035	1/6	167/5.998	51	19.65	3,276,800
1x: [HMOR21]	×	971	433	290	1/0	191/5.236	59	16.95	-
4x	×	1,174	781	355	4/0	185/5.405	226	4.43	-
8x	×	1,679	1,570	532	8/0	182/5.495	445	2.25	-
16x	×	2,587	2,994	855	16/0	172/5.814	840	1.19	-

Table 2: Resource and performance comparison of FrodoKEM-640 Encapsulation.
SC: Standard Compliance

	SC	**LUT**	**FF**	**Slices**	**DSP/ BRAM**	**Freq./Period** (MHz/ns)	**Ops/sec**	Duration (ms)	Cycle Count
This Work	✓	15,908	12,111	5,075	32/20	136.98/7.30	1077	0.928	127,136
[Bus21]	✓	11,396	7,024	-	1/10	167/5.998	50	19.96	3,328,907
[HOKG18]	✓	6,745	3,528	1,855	1/11	167/5.998	51	19.89	3,317,760
1x: [HMOR21]	×	4,246	2,131	1,180	1/0	190/5.263	58	17.24	-
4x	×	4,620	2,552	1,338	4/0	183/5.464	221	4.52	-
8x	×	5,155	3,356	1,485	8/0	177/5.65	427	2.34	-
16x	×	5,796	4,694	1,692	16/0	171/5.85	825	1.21	-

Table 3: Resource and performance comparison of FrodoKEM-640 Decapsulation.
SC: Standard Compliance

	SC	**LUT**	**FF**	**Slices**	**DSP/ BRAM**	**Freq./Period** (MHz/ns)	**Ops/sec**	Duration (ms)	Cycle Count
This Work	✓	17,066	12,868	5,503	32/20	133.15/7.51	1052	0.95	127,567
[Bus21]	✓	11.792	7.451	-	1/10	162/6.17	48	20.76	3,366,126
[HOKG18]	✓	7,220	3,549	1,992	1/16	162/6.17	49	20.72	3,358,720
1x [HMOR21]	×	10,518	2,299	2,933	1/0	190/5.26	57	17.54	-
4x	×	11,581	2,818	3,424	4/0	174/5.75	208	4.81	-
8x	×	13,128	3,737	3,710	8/0	164/6.10	391	2.55	-
16x	×	14,528	5,335	4,020	16/0	160/6.25	763	1.31	-
1x	×	4,466	2,152	1,254	1/12.5	162/6.17	49	20.41	-
4x	×	4,841	2,661	1,345	4/12.5	161/6.21	192	5.21	-
8x	×	5,476	3,479	1,558	8/12.5	156/6.41	372	2.69	-
16x	×	6,881	5,081	1,947	16/12.5	149/6.71	710	1.41	-

matrix A on-the-fly. According to the proposed architecture, it is possible to achieve significantly higher speeds than the example implementation.

Specifically, for key generation, in the $k \times l \times 8$ architecture, the value of k is limited by the SHAKE throughput, whereas l can be increased to scale the performance as desired. Similarly, for encapsulation and decapsulation, in the proposed $8 \times n \times m$ architecture, the value of n depends on the SHAKE speed, while m can be increased without limitation to achieve higher throughput.

Thus, although the example implementation operates around 1 ms, the proposed architecture can be scaled to much higher speeds. Despite concerns about SHAKE limiting throughput in FrodoKEM, parallelization allows for increased operations per second with sufficient resources.

References

[ABD+24] Erdem Alkim, Joppe W. Bos, Léo Ducas, Patrick Longa, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Chris Peikert, Ananth Raghunathan, and Douglas Stebila. FrodoKEM: Learning With Errors Key Encapsulation — Preliminary Standardization Proposal, December 2024. https://frodokem.org/files/FrodoKEM_standard_proposal_20241205.pdf.

[BBC+23] Joppe W. Bos, Olivier Bronchain, Frank Custers, Joost Renes, Denise Verbakel, and Christine van Vredendaal. Enabling FrodoKEM on embedded devices. IACR TCHES, 2023(3):74–96, 2023.

[Bus21] Fabian Buschkowski. Efficient hardware implementation of post-quantum cryptography using high-level synthesis. Master's thesis, Chair for Embedded Security, Ruhr University Bochum, Bochum, Germany, January 2021. Advisor: Georg Land, Supervisor: Prof. Dr.-Ing. Tim Güneysu.

[Fed25] Federal Office for Information Security, Germany. Cryptographic mechanisms: Recommendations and key lengths, January 2025. https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf.

[Fre23] French Cybersecurity Agency. ANSSI Views on the Post-Quantum Cryptography Transition (2023 Follow-Up), December 2023. https://cyber.gouv.fr/sites/default/files/document/follow_up_position_paper_on_post_quantum_cryptography.pdf.

[HMOR21] James Howe, Marco Martinoli, Elisabeth Oswald, and Francesco Regazzoni. Exploring parallelism to improve the performance of FrodoKEM in hardware. Journal of Cryptographic Engineering, 11(4):317–327, November 2021.

[HOKG18] James Howe, Tobias Oder, Markus Krausz, and Tim Güneysu. Standard lattice-based key encapsulation on embedded devices. IACR TCHES, 2018(3):372–393, 2018.

[LBES25] Patrick Longa, Joppe W. Bos, Stephan Ehlen, and Douglas Stebila. FrodoKEM: Key Encapsulation from Learning With Errors. Internet-Draft, IETF, March 2025. https://datatracker.ietf.org/doc/draft-longa-cfrg-frodokem.

[Nat24] National Institute of Standards and Technology. ML-KEM: Module-Lattice-Based Key-Encapsulation Mechanism. Federal Information Processing Standards Publication FIPS 203, National Institute of Standards and Technology, March 2024. Accessed: 2025-06-26.

[Net23] Netherlands National Communications Security Agency. The PQC Migration Handbook, December 2023. https://publications.tno.nl/publication/34641918/oicFLj/attema-2023-pqc.pdf.

[Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In 35th FOCS, pages 124–134. IEEE Computer Society Press, November 1994.